

TINYURL SYSTEM DESIGN

INTRODUCTION

- 1.What is a Tiny URL
- 2.Why do we need a Tiny URL Service
- DECIDING REQUIREMENTS
 - 3.Functional Requirements
 - 4.Non Functional Requirements
- CAPACITY ESTIMATION
 - 5.DAU-MAU
 - 6.Throughput
 - 7.Storage
 - 8.Memory
 - 9.Network and Bandwidth Estimation
- API DESIGN
 - 10.API Design:Generate a Short URL
 - 11.API Design:Get long URL
- HIGH LEVEL DESIGN
 - 12.HIGH LEVEL Design:Generate short URL
 - 13.HIGH LEVEL Design:Problem Collisions
 - 14.HIGH LEVEL Design:Approach 2 Random String Generation using LongURL
 - 15.HIGH LEVEL Design:Approach 3 Check DB for Collisions
 - 16.HIGH LEVEL Design:Approach 4 Let us keep Counters

- 17.HIGH LEVEL Design:Approach 4 Continued Zookeeper
 - 18.HIGH LEVEL Design:Approach 4 Base 62 Encoding
 - 19.HIGH LEVEL Design:Approach 4 Final Design Diagram
 - 20.HIGH LEVEL Design:Get Long URL
 - DEEP DIVE INSIGHTS
 - 21.Deep Dive Insights:Database Selection
 - 22.Deep Dive Insights:Data Modeling
 - 23.Deep Dive Insights:Redirection from short URL
-

INTRODUCTION

What is a Tiny URL Service

Before designing the Tiny URL Service, let's first understand what exactly it is.

A **Tiny URL Service**, also known as a **URL Shortener Service**, helps shorten long URLs.

How It Works:

1. You provide a long URL.
2. The service gives you a shortened URL.



Why Do We Need a Tiny URL Service?

Why Do We Need to Shorten URLs?

1. Easy to Share

Long URL:

<https://www.example.com/products/electronics/phones/smartphones/new-model?color=black&storage=128GB>

Short URL:

<https://tiny.url/xyz123>

Imagine trying to share the long URL above on social media. It's cumbersome and prone to errors. A short URL, on the other hand, is simple to share and remember.

2. Character Limits

Long URL Tweet:

Check out this new smartphone!

<https://www.example.com/products/electronics/phones/smartphones/new-model?color=black&storage=128GB> #tech #newrelease

Short URL Tweet:

Check out this new smartphone! <https://tiny.url/xyz123> #tech #newrelease

Platforms like Twitter have character limits. Short URLs save space, so you can write more.

3. Clean and Professional

Long URL Email:

Hello Team,

Please check the report here:

<https://www.example.com/reports/2024/Q1/financials/summary?>

Best, John

Short URL Email:

Hello Team,

Please check the report here: <https://tiny.url/reportQ1>.

Best, John

The short URL looks cleaner and more professional in emails or documents, improving the overall presentation and readability.

DECIDING REQUIREMENTS

Functional Requirements

Generate a Short URL

- The service takes a long URL as input and returns a shorter version of it.

Get Long URL

- The service takes a short URL as input and returns the original long URL.

Non-Functional Requirements

Requirement	Description
Availability	If the Tiny URL Service is down, people can't open the long URLs. The system must be highly available with 99.99999% uptime.
Low Latency	If the Tiny URL Service is slow, people will experience delays in opening the website. Therefore, we need to have low latency.

Requirement	Description
Scalability	This service will be used to create short URLs for many webpages and will be used by millions of users. We expect large-scale usage, so the system must be scalable to handle the high traffic efficiently.
Get Long URL	The service should take a short URL as input and return the original long URL.

CAPACITY ESTIMATION

DAU MAU

How many users will be using your software?

- **Daily Active Users (DAU):** 300 million
- **Monthly Active Users (MAU):** 1 billion

Throughput

Write Operations

How many writes are performed in the system?

There are only two possible ways to write (add data) into the system based on the functional requirements:

1. Create a short URL for a long URL input

Assumptions:

- 10% of the Daily Active Users (DAU) create short URLs.
DAU = 300 million
10% of 300 million = **30 million users create short URLs every day**
- Each user creates an average of 5 short URLs per day.

Calculations:

30 million users × 5 requests per user per day = 150 million shorten URL requests per day

Read Operations

How many reads are performed in the system?

There is only one way to read data from the system:

1. Getting the long URL from short URL input

Assumptions:

- All daily active users send requests to get long URLs.
- Each user sends an average of 20 "get long URL" requests per day.

Calculations:

300 million DAU × 20 requests per user per day = 6 billion get long URL requests per day

Storage

URL Mappings Data

When a user sends a request to create a short URL, a mapping between **shortURL** and **longURL** is created in the backend.

Storage Estimation

- **Size of the Mapping:**
 - 100 bytes (average size of a long URL)
 - 30 bytes (average size of a short URL)
 - 70 bytes (additional metadata such as timestamp, user information, etc.)
 - **Total per mapping: 200 bytes**

Throughput Estimation

From throughput estimation, we know there are **150 million 'create short URL' requests per day**.

Storage Requirements Calculation

- **Total storage per day:**
(150 million requests * 200 bytes = 30 GB/day)
- **Total storage for 10 years:**
(30 GB/day * 365 * 10 = 109.5 TB)

Summary

- **Daily Storage Requirement:** 30 GB/day
 - **10-Year Storage Requirement:** 109.5 TB
-

Memory

By memory, we mean **cache memory size**.

Importance of Cache

Accessing data from the database takes a long time. However, if we want to access data faster, we use a **cache** to store frequently accessed data temporarily.

Storage Requirements

- The total storage required every day is **30 GB**.
- **30 GB** is manageable, so we can keep all this data in the cache for faster access.

Scalability

The memory size should also scale as our system grows to accommodate increasing storage needs efficiently.

Network and Bandwidth

Network/Bandwidth estimation tells us how much data flows in and out of our system per second.

Data Flow Into Our System (Ingress)

Incoming data in a day is ultimately saved in storage.

Given:

- Data stored in a day = **30 GB/day**

Calculation:

- Incoming data per second: $30 \text{ GB} / (24 \times 60 \times 60) = \mathbf{0.35 \text{ MB/s}}$

Result:

- **Ingress Data Flow = 0.35 MB/s**

Data Flow Out of Our System (Egress)

Outgoing data refers to the data read from our system.

Given:

- **6 billion read (get long URL) requests per day**
- Average size of a read record (URL mapping) = **200 bytes**

Calculation:

- Outgoing data per day: $6 \text{ billion} \times 200 \text{ bytes} = \mathbf{1200 \text{ GB/day}}$
- Outgoing data per second: $1200 \text{ GB} / (24 \times 60 \times 60) = \mathbf{13.8 \text{ MB/s}}$

Result:

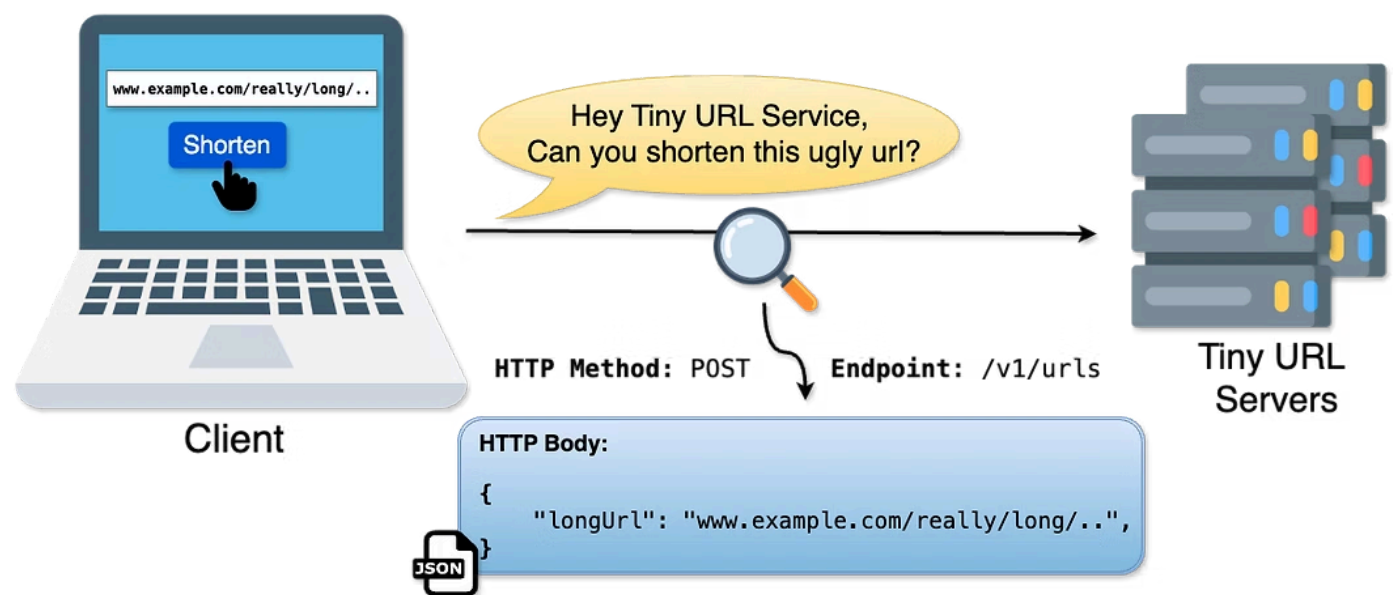
- **Egress Data Flow = 13.8 MB/s**
-

API DESIGN :Generate a Short URL

Let's zoom into the 'communication' for creating a shortened URL from a long URL.

When we ask the server to shorten a long URL, we use an API call. This is how computers communicate with each other.

We will use a **REST API** for this communication. Below are the technical details:



HTTP Method

This tells the server what action to perform. When we ask the server to shorten a URL, we're asking it to create a short URL for the long one we provide. Because we're creating something new, we use the **POST** action.

Endpoint

This tells the server where to perform the action.

Since we are creating a short URL for the long URL input, we will use the **/v1/urls** endpoint of the server.

Note

- **'v1'** refers to version 1. It is a good practice to version your APIs.

Example to Understand HTTP Method and Endpoint

Let's say we are designing an API for creating a new user account:

- **What to do?** → **POST** (action)
- **Where to do it?** → **/v1/users** (endpoint)

HTTP Body

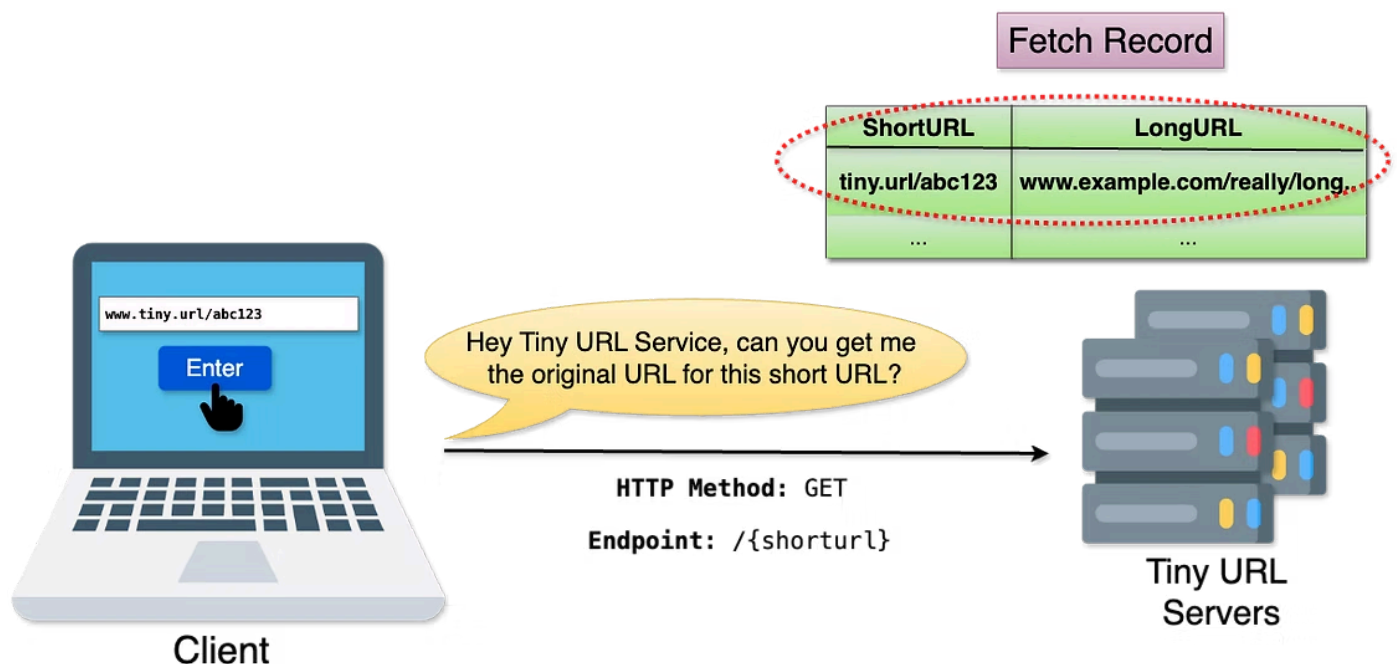
We have told the server to shorten the long URL, but we haven't specified which URL to shorten. This information is sent in the request body:

```
{  
  "longUrl": "Your long URL here"  
}
```

API DESIGN :Get Long URL

Let's zoom into the 'communication' for getting the long URL from the short URL.

We will use a REST API for this communication. Below are the technical details:



HTTP Method

This tells the server what action to perform. Since we want to **fetch** the long URL from the short URL input, we use the **GET** method.

Endpoint

This tells the server where to perform the action. We are fetching long url record (using the short url input), that is why we use the endpoint **/shortUrl**.

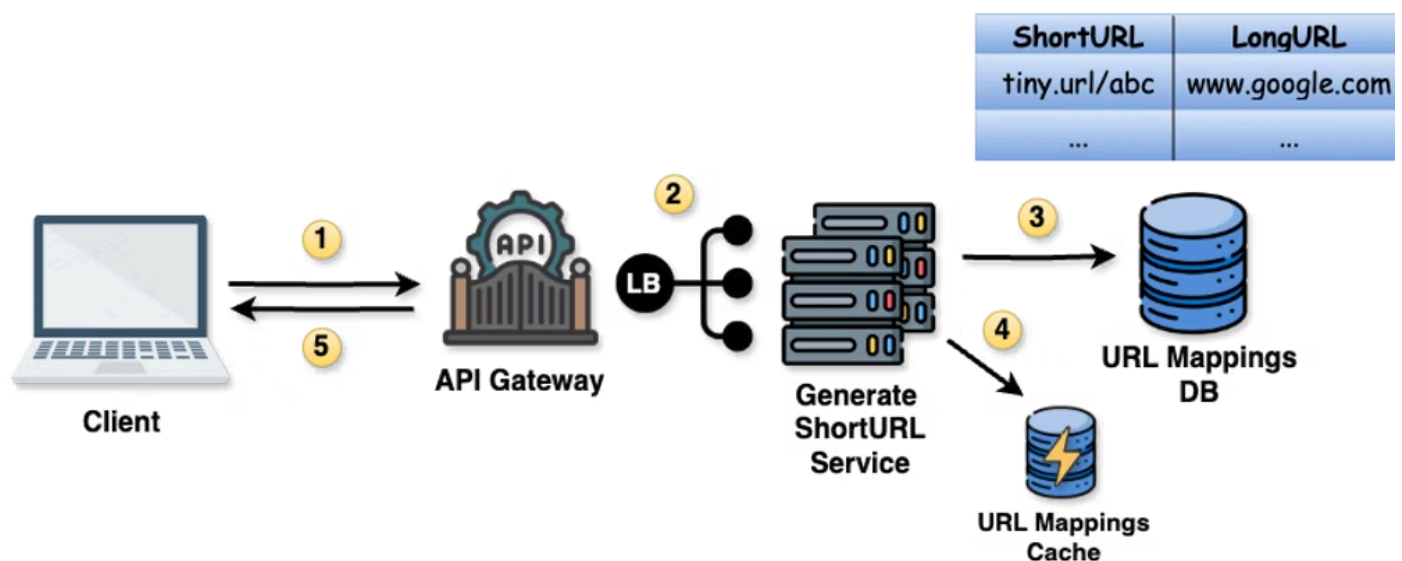
Note

GET requests do not include a body because they are used to **fetch information**, not to **send data**.

HIGH LEVEL DESIGN

HIGH LEVEL DESIGN :Generate Short URL

Let's look at the end-to-end flow for generating a short URL for a long URL. The following steps are involved, corresponding to the numbers in the diagram:



1. **POST Request:**

When the user tries to generate a short URL for a given long URL, a **POST** request is sent with the long URL input in the body.

2. **API Gateway:**

The API Gateway handles the incoming request and routes it to the Generate ShortURL Service via the Load Balancer.

3. **Generate ShortURL Service:**

The Generate ShortURL Service generates a short URL for the long URL input and then saves the mapping (**shortUrl** → **longUrl**) in the database (URL Mappings DB).

4. **Cache Update:**

The URL Mappings Cache is also updated for faster access.

5. **Response:**

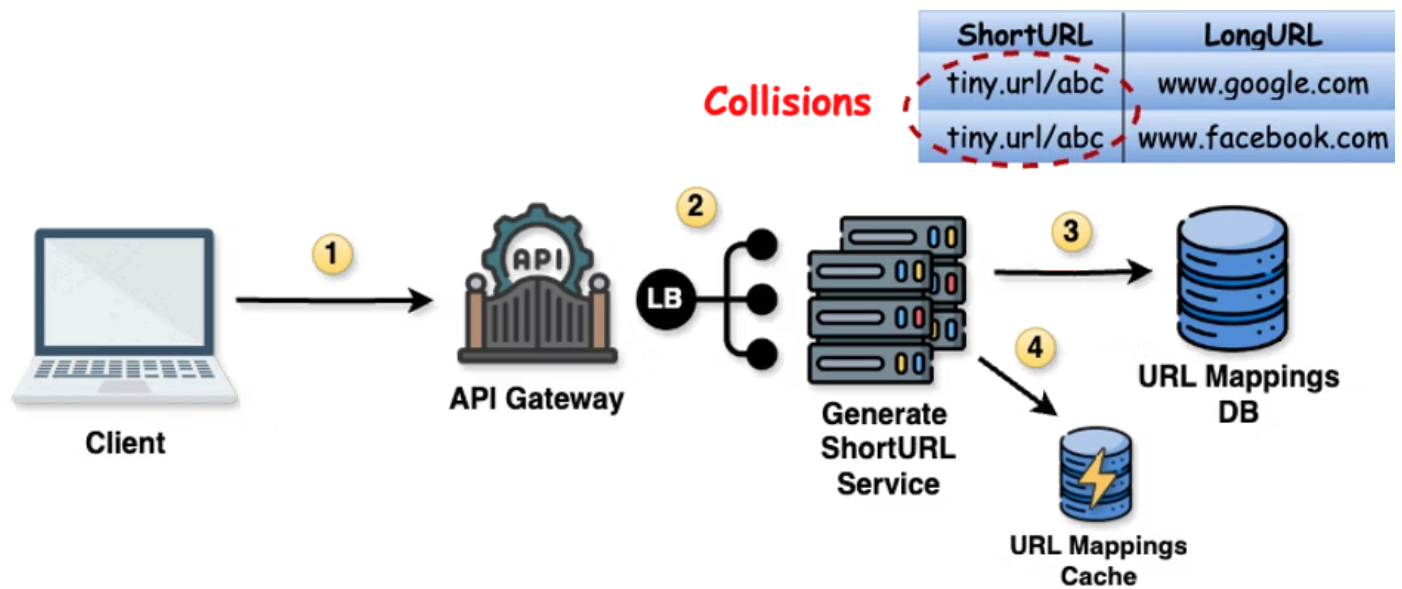
The generated short URL is returned back to the client.

HIGH LEVEL DESIGN :Problem Collisions

Let's consider a scenario when:

- User submits **www.google.com** and gets the short URL **tiny.url/abc**.
- User again submits **www.facebook.com** and also gets the short URL **tiny.url/abc**.

This situation is called a **collision**.



It happens when two different long URLs get the same short URL.

Collisions can cause problems because the system will get confused about which original URL to open when someone clicks on the short URL.

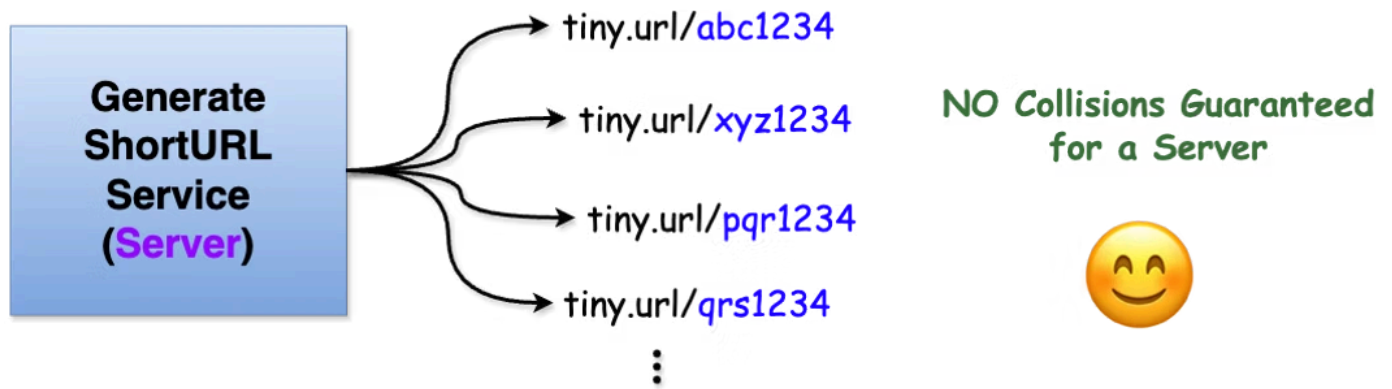
HIGH LEVEL DESIGN: Approach 1- Random String Generation

So how do we avoid collisions?

Collisions happen when the same short URL is generated for two or more different long URLs. To avoid this, we need our short URL generator service to always create a unique random string.

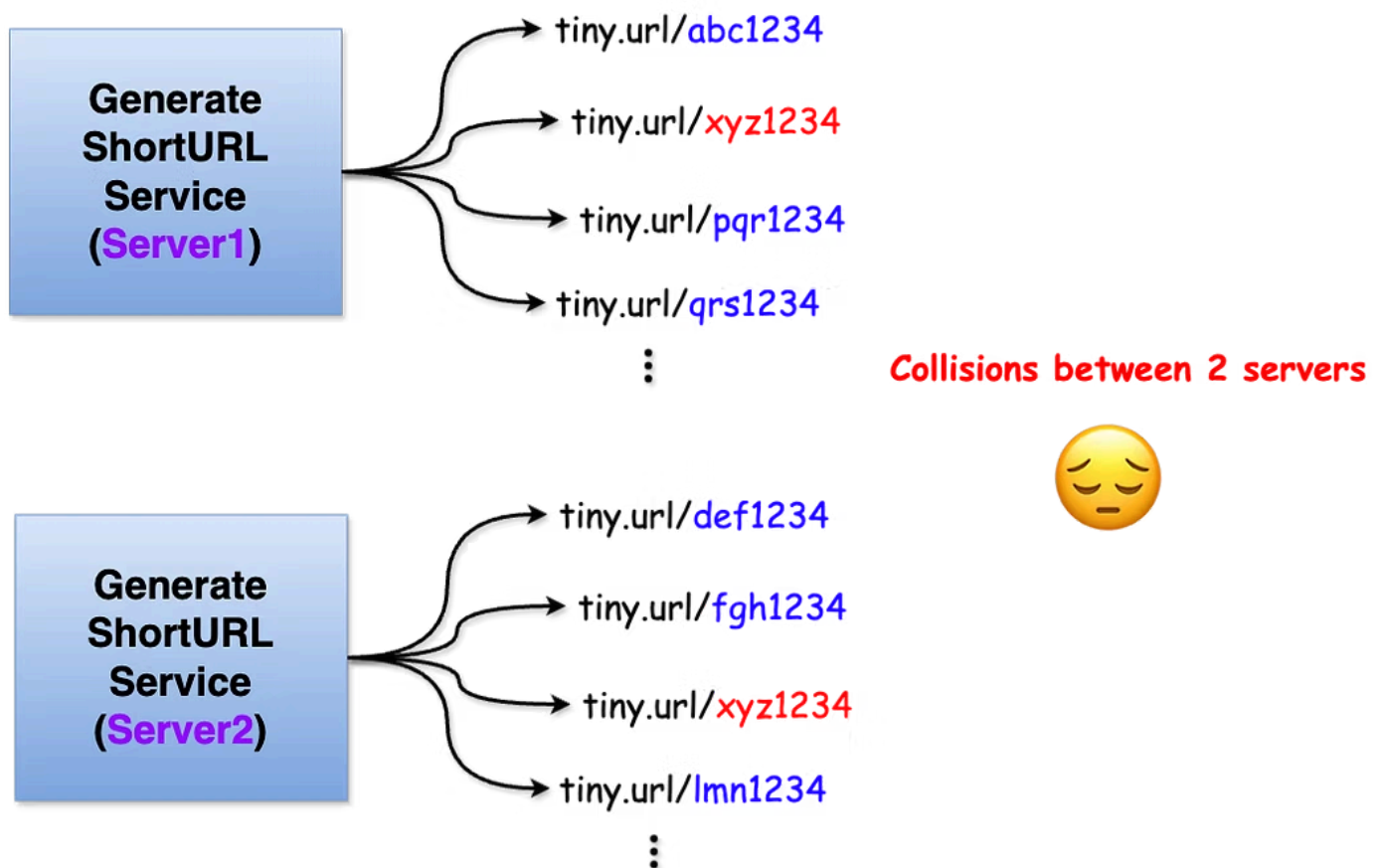
How Do We Achieve This?

There are many algorithms designed to generate unique random strings. These algorithms ensure that every time they run on a server, the generated strings are unique, so there are no collisions.



The Challenge with Multiple Servers

However, when we run the same random string algorithm on two different servers independently, it can't guarantee uniqueness. Collisions can occur because the servers don't know about the strings generated by each other.



Conclusion

So, we can't use this approach to avoid collisions.

HIGH LEVEL DESIGN: Approach 2

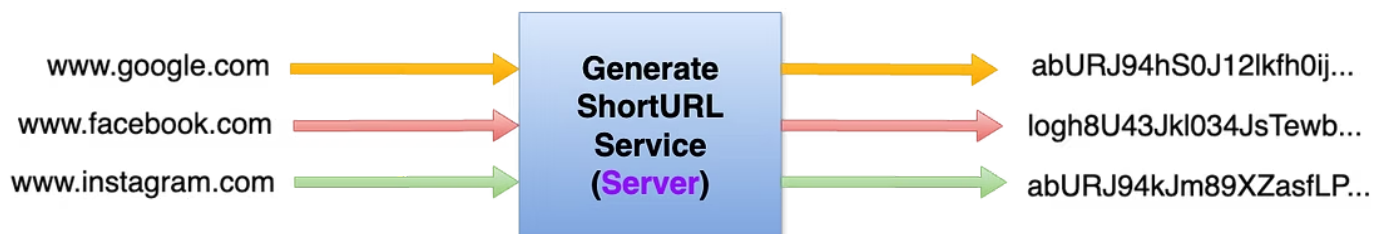
Random String Generation using LongURL

Earlier, we saw that using random string algorithms independently on different servers led to collisions.

To solve this, we can use an algorithm that generates unique random strings, even on different servers. This is where the MD5 algorithm comes in.

How MD5 Works?

- **MD5 doesn't rely on the server but on the input.**
- If we provide different inputs, MD5 guarantees different outputs.
- So, if we run MD5 on two different long URLs, it will produce two unique short URLs with no collisions.



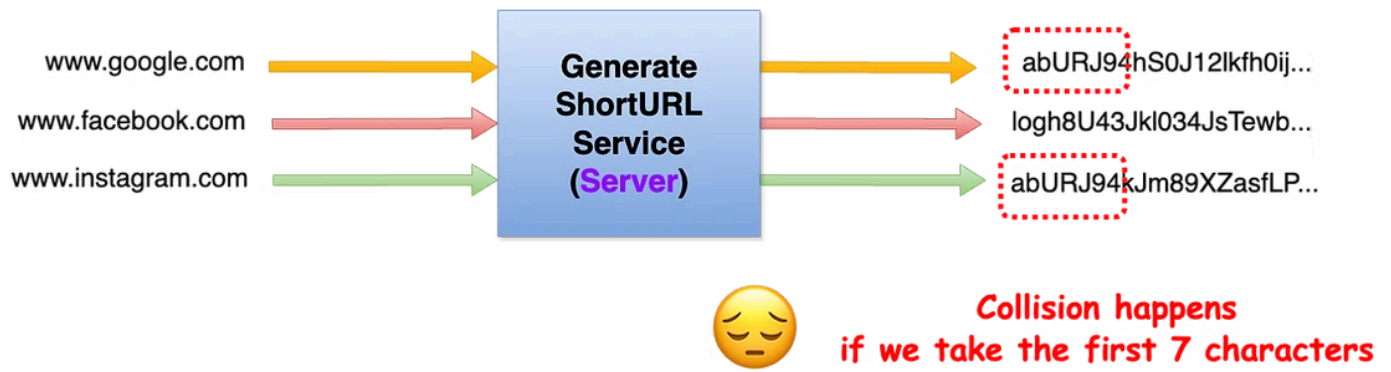
It seems like we solved the problem, right? Actually, no. The output string of MD5 is very long (more than 20 characters). We need a short URL, and more than 20 characters is too long, so this doesn't solve our short URL problem.

Potential Fix

One idea is to take, let's say, the first 7 characters from the MD5 output. Will this work?

No!

- MD5 guarantees that the entire output string is unique but does not guarantee that the first 7 characters will be unique.
- So, collisions can still happen.



Conclusion

Therefore, we can't use this approach either.

HIGH LEVEL DESIGN :Approach 3

Check DB for Collisions

In the previous approaches, we faced collisions while generating random strings. This time, let's allow collisions to happen and handle them as they occur.

Here's what we mean:

If a collision happens, we simply run the algorithm again to generate a new random string.

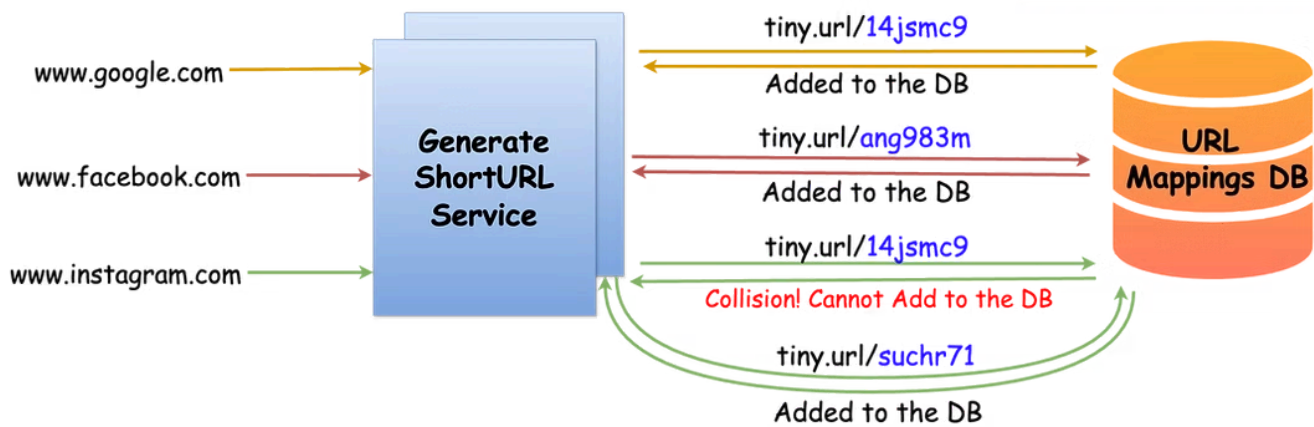
Example:

- **google.com** generates -> **14jsmc9**
- **facebook.com** generates -> **ang983m**
- **instagram.com** generates -> **14jsmc9** (Collision)

Since we have a collision for **instagram.com**, we generate a new random string -> **suchr71**.

So, our approach is to keep generating new strings until we don't get a collision. We check the database to see if a collision happened.

ShortURL	LongURL
tiny.url/14jsmc9	www.google.com
tiny.url/ang983m	www.facebook.com
tiny.url/suchr71	www.instagram.com



This solves the problem but introduces another issue.

Performance Concern:

- It takes a lot of time. Every time a collision happens, we need to check the database, which is slow.
- As our users increase and we generate more short URLs, the chances of collisions also increase, making us slower.

Since we need **low latency** (as per our non-functional requirements), we can't use this approach.

HIGH LEVEL DESIGN :Approach 4 Let us Keep Counters

So far, we have seen that using random number generation leads to collisions. When collisions occurred, we regenerated the string until there were no collisions.

Can We Avoid Collisions Entirely Without Regeneration?

Let's try a simple numbering system.

Example

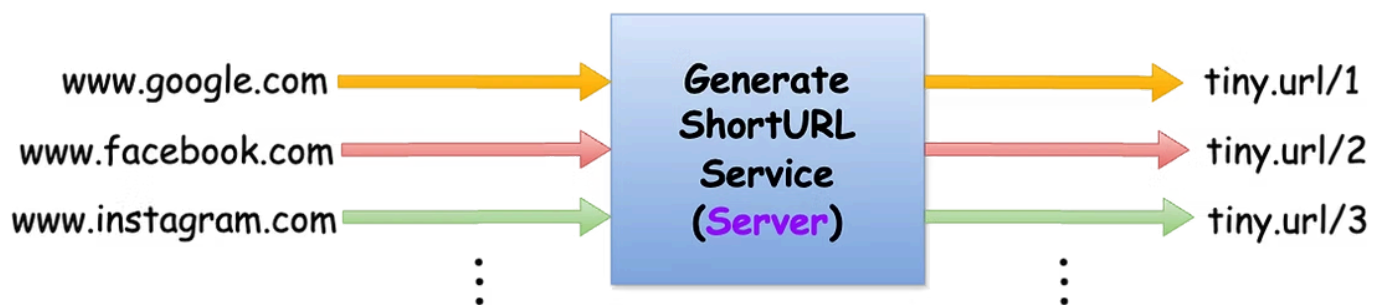
google.com -> tiny.url/1

facebook.com -> tiny.url/2

instagram.com -> tiny.url/3

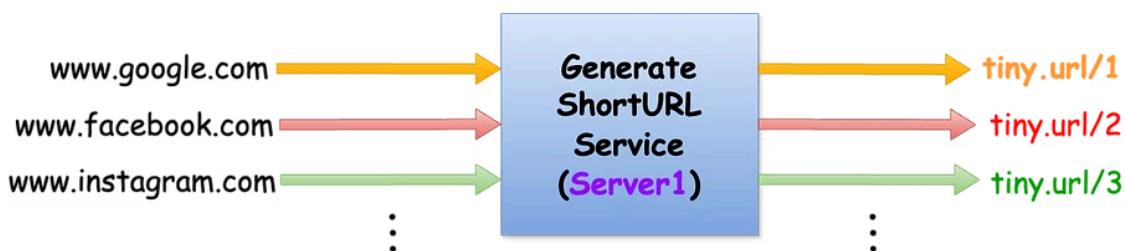
...

This way, we assign a unique number to each long URL input, and collisions can never happen.

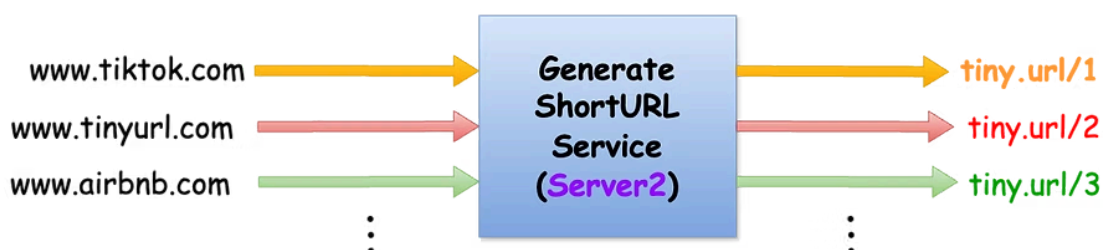


Problem with Independent Servers

However, when we run this algorithm independently on different servers, the same issue arises. Each server does not know about the other servers and will start generating numbers from 1, 2, 3, 4, and so on. This leads to collisions again!



Collisions! 🙄



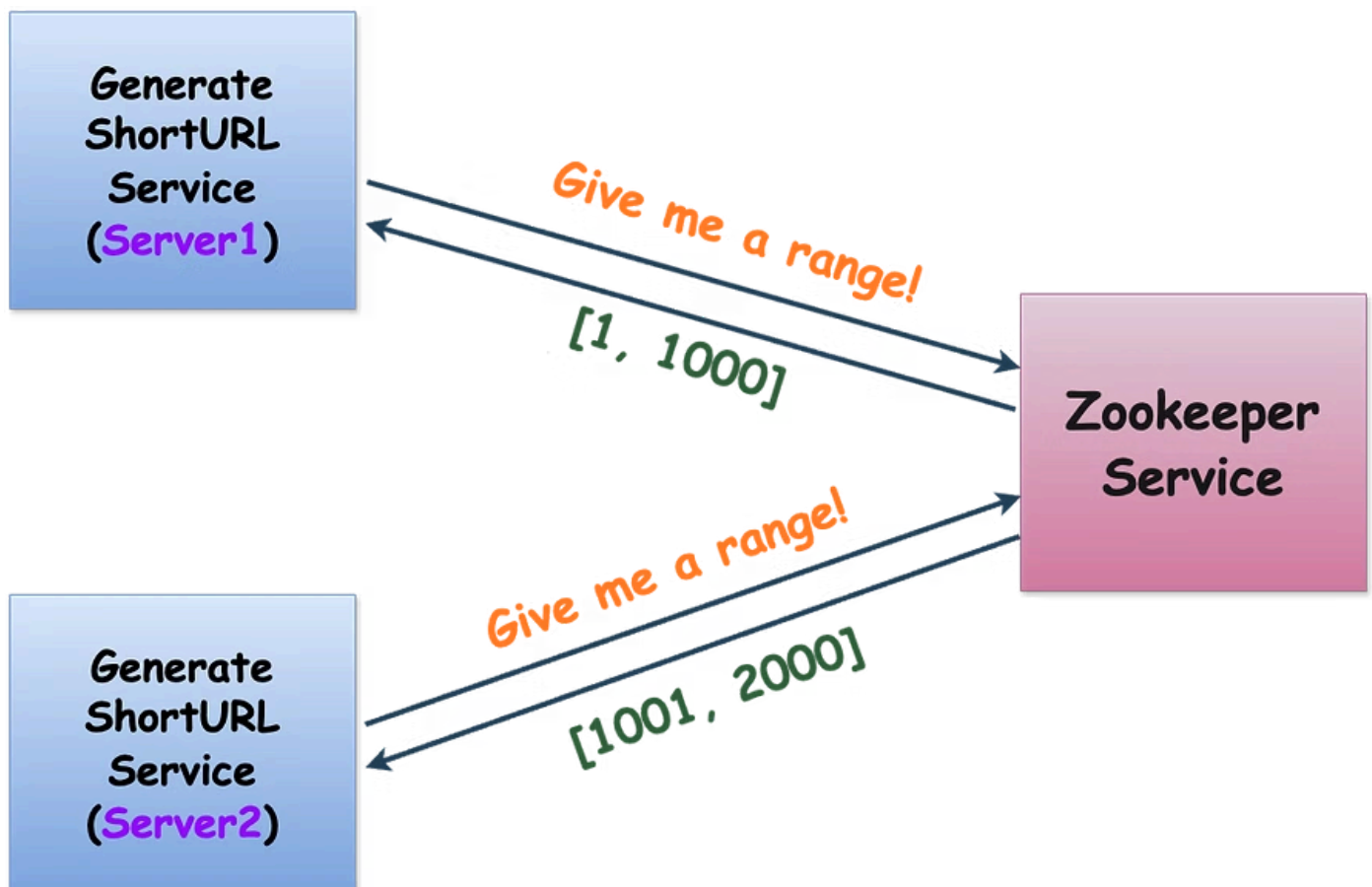
To solve this, we need a way for servers to coordinate with each other to avoid collisions.

ZooKeeper solves this exact problem. Interested in learning how? Let's explore this next.

HIGH LEVEL DESIGN :Approach 4 Continued :ZooKeeper

ZooKeeper helps coordinate between all the servers to make sure there are no collisions in the generated URLs.

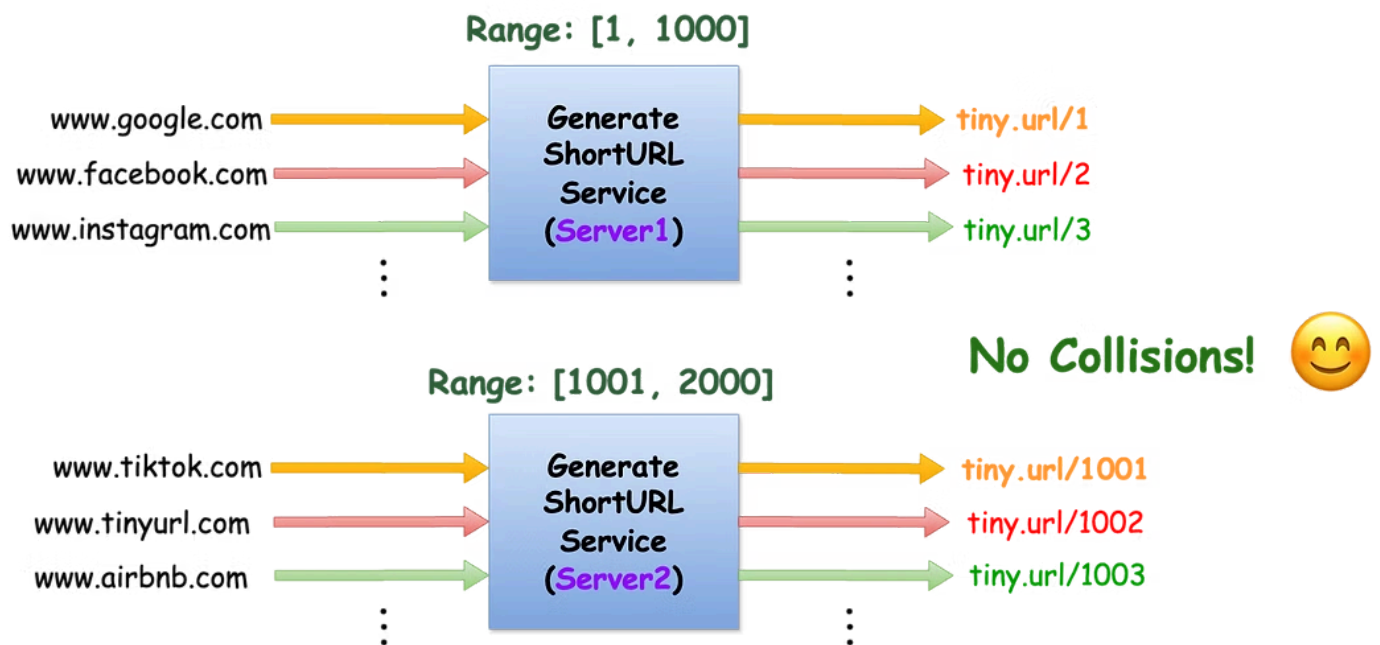
- Each server is connected to ZooKeeper and can communicate with it.
- Every server asks ZooKeeper for a range of numbers. ZooKeeper gives each server a different range to avoid collisions.



- The server then uses this range to generate numbers one by one.

Example

- Server 1 gets the range **[1, 1000]** from ZooKeeper.
- Server 2 gets the range **[1001, 2000]** from ZooKeeper.



This way, ZooKeeper brings coordination among all servers and solves the collision problem completely.

HIGH LEVEL DESIGN :Approach 4 Base 62 Encoding

Note: The following text might seem overwhelming at first, but don't worry. Take your time to read it once or twice patiently, and it will make sense.

So far, we have seen that Zookeeper sends a range to each server to prevent collisions.

Let's follow this numbering and see how far we can go:

tiny.url/1, ... tiny.url/10, ... tiny.url/1000, ... tiny.url/9876549, ...

As we keep going, the length of our URLs increases, but our goal is to generate **short** URLs.

What can we do about this?

Right now, each digit is in **Base 10 (0-9)**.

How about we make each digit a value from **0–9 (10)**, **A–Z (26)**, and **a–z (26)** = **Base 62 (10 + 26 + 26)**?

What does this mean? Let's understand better:

Using **(0-9)**, we generated URLs like:

tiny.url/1, ... **tiny.url/10**, ... **tiny.url/1000**, ... **tiny.url/9876549**, ...

Similarly, if we use **(0-9, a-z, A-Z)**, our URLs look like:

tiny.url/1, ..., **tiny.url/a**, ..., **tiny.url/g8**, ..., **tiny.url/fRLB** (which represents **9876549**), ...

How did we generate these?

Let's take an example of **9876549**.

In Base 10, **9876549** is written as:

$$(9 + 10^6) + (8 \times 10^5) + (7 \times 10^4) + (6 \times 10^3) + (5 \times 10^2) + (4 \times 10^1) + (9 \times 10^0)$$

In Base 62, 9876549 is written as fRLB:

$$\text{fRLB} = (f \times 62^3) + (R \times 62^2) + (L \times 62^1) + (B \times 62^0) = (41 \times 62^3) + (27 \times 62^2) + (21 \times 62^1) + (11 \times 62^0)$$

See the Base 62 table for reference below:

Decimal	Binary	Base62	Decimal	Binary	Base62	Decimal	Binary	Base62	Decimal	Binary	Base62
0	000000	0	16	010000	G	32	100000	W	48	110000	m
1	000001	1	17	010001	H	33	100001	X	49	110001	n
2	000010	2	18	010010	I	34	100010	Y	50	110010	o
3	000011	3	19	010011	J	35	100011	Z	51	110011	p
4	000100	4	20	010100	K	36	100100	a	52	110100	q
5	000101	5	21	010101	L	37	100101	b	53	110101	r
6	000110	6	22	010110	M	38	100110	c	54	110110	s
7	000111	7	23	010111	N	39	100111	d	55	110111	t
8	001000	8	24	011000	O	40	101000	e	56	111000	u
9	001001	9	25	011001	P	41	101001	f	57	111001	v
10	001010	A	26	011010	Q	42	101010	g	58	111010	w
11	001011	B	27	011011	R	43	101011	h	59	111011	x
12	001100	C	28	011100	S	44	101100	i	60	111100	y
13	001101	D	29	011101	T	45	101101	j	61	111101	z
14	001110	E	30	011110	U	46	101110	k			
15	001111	F	31	011111	V	47	101111	l			

Using **0-9, a-z, and A-Z** allows us to create many short URLs with less space. This is because each character can be one of 62 possibilities, while in Base 10, it would only be one of 10 (0-9).

In fact, with **7 characters**, we can generate:

62^7 ~ 3.5 trillion URLs

This is more than enough for our scale.

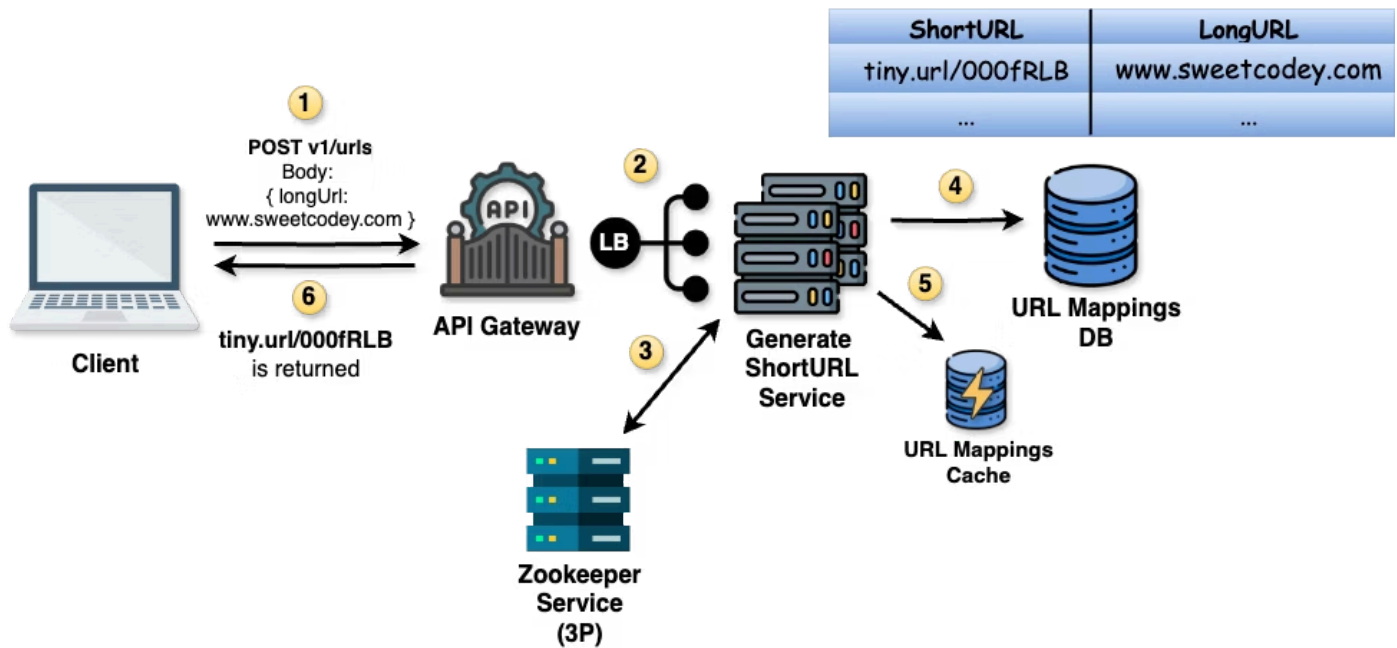
Therefore, by using **Base 62**, we can generate a large number of short URLs efficiently.

HIGH LEVEL DESIGN :Approach 4 :Final Design Diagram

To sum up, by adding the Zookeeper service and using Counters, we have completely solved the collision problem and also made our short URL generation process more efficient.

Final Flow

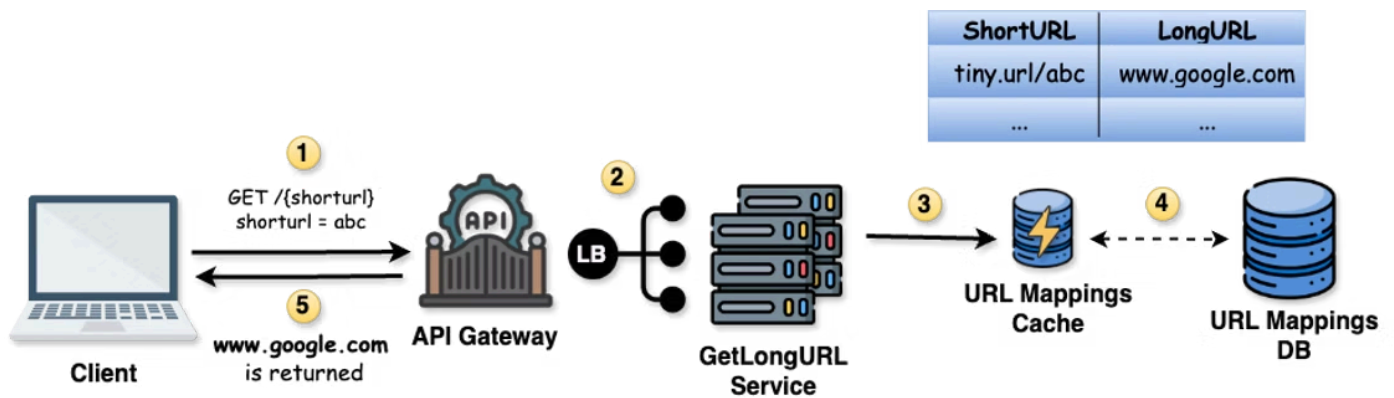
The final flow looks as follows. The following steps occur corresponding to the numbers in the diagram:



1. When the user tries to generate a short URL for a given long URL, a **POST** request is sent with the long URL input in the body.
2. The API Gateway handles the incoming request and routes it to the Generate ShortURL Service via Load Balancer.
3. Generate ShortURL Service gets the range (aka counters) from Zookeeper Service.
4. Generate ShortURL Service generates a short URL for the long URL input using a counter. It then saves the mapping (**shortUrl** → **longUrl**) in the database (URL Mappings DB).
5. URL Mappings Cache is also updated for faster access.
6. The generated short URL is returned back to the client.

HIGH LEVEL DESIGN: Get Long URL

Let's look at the end-to-end flow for getting the long URL. The following steps are involved, corresponding to the numbers in the diagram:



1. User Request

The user types in `shorturl` in the browser. When we press enter, the browser sends a `GET` request to the API Gateway on the `/{{shorturl}}` endpoint.

2. API Gateway Forwarding

The API Gateway receives the request and forwards it to the `GetLongURL` Service via the Load Balancer.

3. Cache Lookup

The `GetLongURL` Service checks the cache (URL Mappings) for the short URL.

4. Database Query (if Cache Miss)

If the short URL is not found in the cache, the cache (using a read-through mechanism) automatically queries the URL Mappings DB to get the long URL and updates the cache with this information for future requests.

5. Response to User

The `GetLongURL` Service returns the long URL (e.g., `www.google.com`) to the API Gateway, which then sends it back to the user.

DEEP DIVE INSIGHTS

DEEP DIVE INSIGHTS: Database Selection

In order to decide the DB type, here are some general guidelines to follow. However, remember that it's not always black and white — the decision heavily depends on the project requirements.

Guidelines

- **Fast Data Access:**

NoSQL is generally preferred over SQL for faster access.

- **Handling Large Scale:**

NoSQL databases tend to perform better than SQL databases when handling a large scale.

- **Data Structure:**

- SQL is more suited when data fits into a fixed structure.
- NoSQL is better when the data doesn't fit into a fixed structure.

- **Complex Queries:**

- SQL is recommended for complex queries.
- NoSQL works well for simpler queries.

- **Data Evolution:**

NoSQL is ideal if your data changes frequently or will evolve over time, as it supports a flexible structure.

Database Decision Example

Database	Deciding Factors	Decision
URLMappingsDB	• Quick access needed to fetch the long URL for the short URL to ensure faster website access. • Large scale data handling (millions of URL mappings per day). • Simple query pattern (read URL mappings using the short URL as the key).	NoSQL

DEEP DIVE INSIGHTS: Data Modeling

URL Mappings Schema



URL Mappings DB

ShortURL	LongURL
tiny.url/00000G9	www.tiktok.com
tiny.url/00000GA	www.twitter.com
tiny.url/000fRLB	www.sweetcodey.com

Database Type

- NoSQL

Common Queries

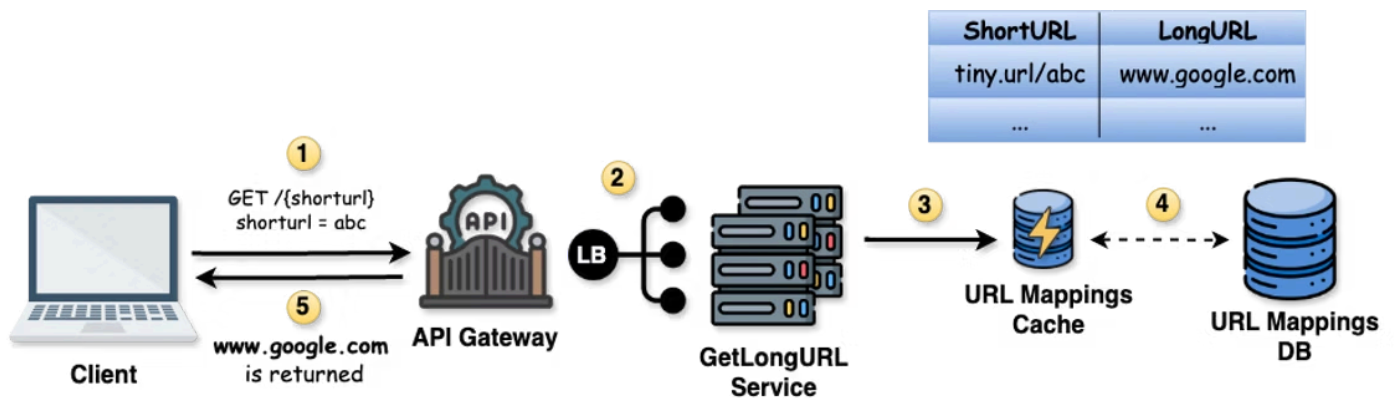
- Read URL mappings using the short URL as the key.

Indexing

- We will be using a **key-value store** (a type of NoSQL) for the URL Mappings database.
- The database is **automatically indexed on the key**, which is the short URL.
- This allows for **fast lookups and retrievals**.

DEEP DIVE INSIGHTS :Redirection from Short URL

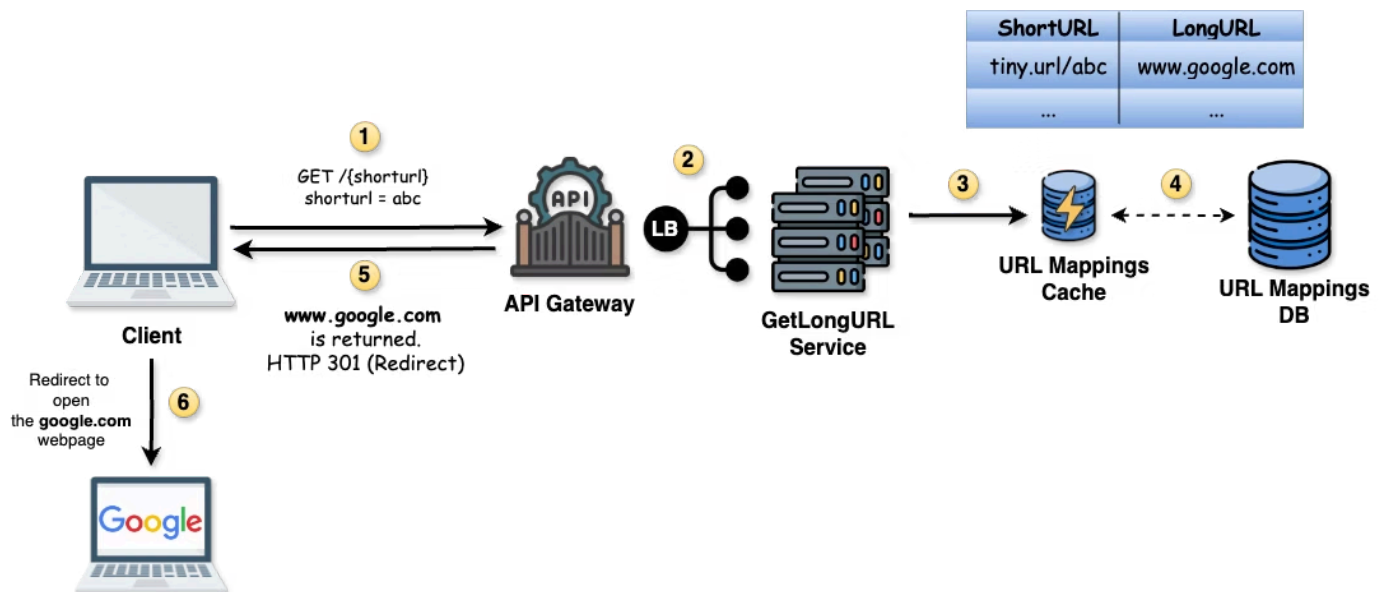
Previously, we saw the end-to-end flow for getting the long URL when the user sends a **GET** request to the `/ {shorturl}` endpoint.



However, in reality, it works a bit differently. The story doesn't end at step 5. Just returning www.google.com back to the client is not sufficient. This is because the browser needs to open the webpage for the long URL.

Understanding the Flow

Let's break down how this actually works:



1. User Input:

- The user types the short URL in the browser and presses enter.
- The browser sends a **GET** request to the API Gateway on the `/ {shorturl}` endpoint.

2. Request Handling:

- The API Gateway receives the request and forwards it to the **GetLongURL Service** via the Load Balancer.

3. **Cache Lookup:**

- The **GetLongURL Service** checks the cache (URL Mappings) for the short URL.

4. **Database Query (if Cache Miss):**

- If the short URL is not found in the cache, the cache (using a read-through mechanism) automatically queries the **URL Mappings DB** to get the long URL.
- The cache is updated with this information for future requests.

5. **Response from GetLongURL Service:**

- The GetLongURL Service returns the long URL (www.google.com) to the API Gateway.

6. **Redirection:**

- The API Gateway sends the response back to the user with the **HTTP Status Code 301**.
- This code instructs the browser to redirect to the long URL webpage.

7. **Browser Action:**

- The browser opens the long URL webpage.